



Felvételi, 2022. szeptember
– Alapképzés, informatika vizsga –

Hivatalból 1 pont jár. Munkaidő 3 óra.

Megjegyzések:

- A pszeudokód algoritmusban
 - az „=” értékadást jelent
 - a „**minden** $i = 1, n, 1$ **végezd**” jelentése: minden $i = 1, 2, 3, \dots, n$ értékre végezd el...
 - a „**minden** $i = n, 1, -1$ **végezd**” jelentése: minden $i = n, n-1, n-2, \dots, 1$ értékre végezd el...
- Minden egész érték tárolható `integer/int` típusú változóban.
- A bemeneti adatok helyesnek tekinthetők (például, ha a feladat szövege azt mondja, hogy $1 < n < 1001$, akkor ez megelőlegezhető, és nem kell leellenőrizni).
- A programokat megjegyzésekkel kell ellátni!
- Törekedjünk hatékony megoldásokra! (maximális pontszám feltétele)

1. (1 pont) Mi kerül kiírásra az alábbi pszeudokód algoritmusrészlet nyomán, amennyiben $n = 8$?

```
c = 0
amíg n > 0 végezd
    a = n + 1
    b = n - 1
    c = c + a + b
    n = n - 1
```

kiír c

Megoldás: 72

2. (1 pont) Mit ír ki az alábbi pszeudokód algoritmusrészlet, ha $n = 5$ és $a[1..n] = \{-1, 6, -9, -2, 5\}$?

```
p1 = 0
p2 = 0
minden i = 1, n, 1 végezd
    ha a[i] > 0 akkor
        p1 = p1 + a[i]
    különben
        p2 = p2 - a[i]
```

kiír p1 + p2

Megoldás: 23

3. (1 pont) Írj Pascal vagy C/C++ **programot**, amely beolvas a billentyűzetről két, pontosan kétszámjegyű természetes számot, majd kiírja az első szám számjegyeit növekvő, a második szám számjegyeit pedig csökkenő sorrendben, külön-külön sorba, egy-egy szóközzel elválasztva egymástól.

PÉLDA

INPUT

13 47

OUTPUT

13

74

Megoldás:

```
#include <iostream>
using namespace std;
int main() {
    int number1, number2;
    cin >> number1 >> number2;
    if( number1 / 10 < number1 % 10 )
    {
        cout << number1 / 10 << " " << number1 % 10;
    } else
    {
        cout << number1 % 10 << " " << number1 / 10;
    }
    cout << endl;
    if( number2 / 10 > number2 % 10 )
    {
        cout << number2 / 10 << " " << number2 % 10;
    } else
    {
        cout << number2 % 10 << " " << number2 / 10;
    }
    return 0;
}
```

4. (1 pont) Írj Pascal vagy C/C++ **programot**, amely beolvas a billentyűzetről két nullától különböző természetes számot. Az első tekintjük egy időszámításunk előtti évszámnak, a másodikat pedig egy időszámításunk szerinti évszámnak. A program írja ki külön sorokba a két évszám közötti összes évszámot (Megjegyzés: nincs 0. év).

PÉLDA

INPUT

5 3

OUTPUT

i.e. 5

i.e. 4

i.e. 3

i.e. 2

i.e. 1

i.sz. 1

i.sz. 2

i.sz. 3

Megoldás:

```
#include <iostream>
using namespace std;
int main() {
    int number1, number2;
    cin >> number1 >> number2;
    for( int i = number1; i >= 1; i-- )
        cout << "i.e. " << i << endl;
    for( int i = 1; i <= number2; i++ )
        cout << "i.sz. " << i << endl;
    return 0;
}
```

5. (1 pont) Írj Pascal vagy C/C++ **programot**, amely pozitív egész számokat olvas be a billentyűzetről 0 végjelig, majd kiír egy összeget, amelyet a beolvasott számok számjegyeinek összegéből kapunk (Megjegyzés: egy összeget kell meghatározni, nem számonkénti összegeket).

PÉLDA

INPUT

12 23 34 45 0

OUTPUT

24

Magyarázat: $1 + 2 + 2 + 3 + 3 + 4 + 4 + 5 = 24$

Megoldás:

```
#include <iostream>
using namespace std;
int main() {
    int sum = 0, nr, nr1;
    do
    {
        cin >> nr;
        nr1 = nr;
        while( nr1 )
        {
            sum += nr1 % 10;
            nr1 /= 10;
        }
    } while( nr != 0 );
    cout << sum;
    return 0;
}
```

6. (1 pont) Írj Pascal vagy C/C++ **programot**, amely beolvas a billentyűzetről egy ötszámjegyű természetes számot, és kiírja külön-külön sorba a beolvasott szám számjegyeiből alkotott számokat lépésenként, először balról-jobbra, majd jobbról-balra haladva (lásd a példát).

PÉLDA

INPUT

45975

OUTPUT

4
45
459
4597
45975
5
57
579
5795
57954

Megoldás:

```
#include <iostream>
using namespace std;
int main() {
    int nr, a, b, c, d, e;
    cin >> nr;
    a = nr / 10000;
    b = nr % 10000 / 1000;
    c = nr % 1000 / 100;
    d = nr % 100 / 10;
    e = nr % 10;
    cout << a << endl;
    cout << a * 10 + b << endl;
    cout << a * 100 + b * 10 + c << endl;
```

```

cout << a * 1000 + b * 100 + c * 10 + d << endl;
cout << a * 10000 + b * 1000 + c * 100 + d * 10 + e << endl;
cout << e << endl;
cout << e * 10 + d << endl;
cout << e * 100 + d * 10 + c << endl;
cout << e * 1000 + d * 100 + c * 10 + b << endl;
cout << e * 10000 + d * 1000 + c * 100 + b * 10 + a << endl;
return 0;
}

```

7. (1 pont) Írj Pascal vagy C/C++ **függvényt**, amely paraméterként kap egy n természetes számot és két n elemű, 100-nál kisebb természetes számokat tartalmazó számsorozatot (mindkét számsorozatban az elemek kettőnként különböznek). A függvény eldönti, hogy az első számsorozat a második számsorozat egyik permutációja-e. A függvény térítsen vissza **1**-et, amennyiben a válasz igen, és **0**-át, amennyiben a válasz nem.

PÉLDA

Ha $n=4$

I. számsorozat: 11 28 13 84

II. számsorozat: 11 13 28 84

A válasz **igen**.

Ha $n=4$

I. számsorozat: 1 12 27 4

II. számsorozat: 4 1 12 28

A válasz **nem**.

Megoldás:

```

int test( int n, int a[100], int b[100] )
{
    int st[100] = { 0 };
    for( int i = 0; i < n; i++ ){
        st[a[i]] = 1;
    }
    for( int i = 0; i < n; i++ ){
        if( st[b[i]] == 0 )
            return 0;
    }
    return 1;
}

```

8. (1 pont) Írj Pascal vagy C/C++ **programot**, amely a *matrix.in* állományból beolvassa az n páros természetes számot ($4 \leq n \leq 100$), valamint egy $n \times n$ méretű mátrixot. Tekintsük a mátrix 1. zig-zag-főátlójának a következő számsort: (i) indulunk a bal felső sarokból; (ii) lépünk egyet JOBBRA; (iii) majd a kettőt LE, kettőt JOBBRA minta szerint lépegetünk, míg az utolsó oszlopba nem érünk; (iv) egy utolsó „egyet LE” lépéssel a jobb alsó sarokban kötünk ki. Továbbá, tekintsük a mátrix 2. zig-zag-főátlójának a következő számsort: (i) indulunk a bal felső sarokból; (ii) lépünk egyet LE; (iii) majd a kettőt JOBBRA, kettőt LE minta szerint lépegetünk, míg az utolsó sorba nem érünk; (iv) egy utolsó „egyet JOBBRA” lépéssel a jobb alsó sarokban kötünk ki. A program írja ki a képernyőre, hogy melyik zig-zag-főátló mentén nagyobb az elemek összege. Ha a két főátlón levő számok összege megegyezik, a program írja ki az EGYENLŐ üzenetet.

PÉLDA

INPUT

4

1 2 4 5

8 9 0 5

OUTPUT

1

2 8 6 6

7 3 5 9

Magyarázat: $1 + 2 + 8 + 6 + 9 > 1 + 8 + 0 + 5 + 9$

1	2	4	5
8	9	0	5
2	8	6	6
7	3	5	9

1	2	4	5
8	9	0	5
2	8	6	6
7	3	5	9

Megoldás:

```
#include <iostream>
using namespace std;

int main()
{
    int x, n, s1 = 0, s2 = 0;
    ofstream file("matrix.txt");
    file >> n;
    for( int i = 0; i < n ; ++i ){
        for( int j = 0; j < n ; ++j ){
            file >> x;
            if( i == 0 && j == 0 || i == n - 1 && j == n - 1 ){
                s1 += x; s2 += x;
            }
            if( j % 2 == 1 && (j == i + 1 || j == i - 1) ){
                s1 += x;
            }
            if( i % 2 == 1 && (j == i + 1 || j == i - 1) ){
                s2 += x;
            }
        }
    }
    if( s1 > s2 ){ cout << "1"; }
    else if( s2 > s1 ){ cout << "2"; }
    else { cout << "EGYENLO"; }
    file.close();
    return 0;
}
```

9. (1 pont) Írj Pascal vagy C/C++ **függvényt**, amely paraméterként megkapja az n és m természetes számokat ($1 < n, m < 101$), valamint egy n x m méretű mátrixot (elemei egész számok). A függvény térítse vissza, hogy egy szöcske hány út között választhat, amennyiben oszlopról oszlopra szökdelve szeretne átjutni a mátrixon a következő szabály szerint: minden oszlopban pontosan egy elemet érint, az illető oszlop minimumát.

PÉLDA

INPUT

4 2

1 2

8 9

1 8

7 3

OUTPUT

2

Magyarázat (amennyiben 1-től n-ig és 1-től m-ig indexelünk):

(1,1), (1,2)

(3,1), (1,2)

Megoldás:

```
int numberOfPaths( int n, int m, int a[][100] ){
    int p = 1; //utak szama
    //bejaras oszloponkent
    for( int j = 0; j < m ; ++j ){
        int minValue = a[0][j]; //kurrens oszlop minimuma
        for( int i = 1; i < n ; ++i ){
            if( a[i][j] < minValue ) { minValue = a[i][j]; }
        }
        int minCount = 0; //ennyi minimum van a kurrens oszlopban
        for( int i = 0; i < n ; ++i ){
            if( a[i][j] == minValue ) { ++minCount; }
        }
        p *= minCount;
    }
    return p;
}
```